Shared policies and colimit certificates

Simon Frost

Table of contents

1 Overview 1

2 Two seasons of prescribed burning 2

3 With a rule each year 3

4 One rule for both seasons 3

5 Testing a shared rule 4

6 A colimit certificate for the composite 5

7 Summary 6

1 Overview

A management rule is often written once and applied every year. A burning regulation, a harvest quota or a grazing limit does not change from season to season, and the people who apply it may keep no record of what was done last time. The rule is then a *shared policy*: one table of action probabilities used by several decisions that cannot be told apart.

This vignette takes a two-season prescribed-burning rule and shows three things, following ADRs 0024 and 0025:

1. with a shared rule the best policy can be randomised, and no fixed rule comes close;
2. at that optimum each season on its own would still change its action, so the per-component equilibria of the fourth vignette are the wrong test; what holds is a first-order condition, which `policy_regrets` checks;
3. a colimit certificate records how the two seasons were glued, and the Lean checker confirms that the composite *is* the colimit of its parts, which makes its semantics exactly the composite of theirs.

2 Two seasons of prescribed burning

Each season the land manager decides whether to burn. Burning lowers the fuel load, which lowers the loss from a later wildfire, but burning in both seasons damages the habitat. A burn costs 1, a wildfire on a high fuel load loses 8, and good habitat is worth 10.

- season 1: the decision `Burn1` with no information, its cost, and the fuel load `Fuel1` it leaves;
- season 2: the decision `Burn2`, also with no information, its cost, the fuel load `Fuel2` from `Fuel1` and `Burn2`, the wildfire loss on `Fuel2`, and the habitat from both burns.

`Burn2` does not read `Burn1` or `Fuel1`: the manager keeps no record. Every probability is a dyadic fraction, so that binary64 arithmetic is exact and the certificate at the end can be checked exactly.

```
using CategoricalInfluenceDiagrams

YN = [:no, :yes]
LH = [:low, :high]
HQ = [:poor, :good]

season1 = Open(influence_diagram(:Burn1 ⇒ YN, :Fuel1 ⇒ LH;
                                 mechanisms = [:Fuel1 ⇒ :Burn1],
                                 decisions = [:Burn1 ⇒ Symbol[]],
                                 utilities = [:Cost1 ⇒ :Burn1], closed = false);
               outputs = [:Burn1, :Fuel1])
season2 = Open(influence_diagram(:Burn1 ⇒ YN, :Fuel1 ⇒ LH, :Burn2 ⇒ YN,
                                 :Fuel2 ⇒ LH, :Habitat ⇒ HQ;
                                 mechanisms = [:Fuel2 ⇒ (:Fuel1, :Burn2),
                                               :Habitat ⇒ (:Burn1, :Burn2)],
                                 decisions = [:Burn2 ⇒ Symbol[]],
                                 utilities = [:Cost2 ⇒ :Burn2, :Wildfire ⇒ :Fuel2,
                                              :HabitatValue ⇒ :Habitat],
                                 closed = false);
               inputs = [:Burn1, :Fuel1])
chain = [season1, season2]
nothing # hide
```

The tables. A kernel's table lists its outputs first: column `j` is the distribution of the child given the `j`-th parent state.

```
ax(x, labels) = FiniteAxis(x, labels)
# A two-parent table from P(child = first state | parents), parents in axis order.
two_parent(p) = permutedims(cat(p, 1 .- p; dims = 3), (3, 1, 2))
kernels = Dict{Symbol,FiniteKernel}(
    :Fuel1 ⇒ FiniteKernel(FiniteSpace(ax(:Burn1, YN)), FiniteSpace(ax(:Fuel1, LH)),
                          [0.25 0.75; 0.75 0.25]),
    :Fuel2 ⇒ FiniteKernel(FiniteSpace([ax(:Fuel1, LH), ax(:Burn2, YN)]),
                          FiniteSpace(ax(:Fuel2, LH)),
                          two_parent([0.5 0.875; 0.125 0.75])),
```

```
    :Habitat ⇒ FiniteKernel(FiniteSpace([ax(:Burn1, YN), ax(:Burn2, YN)]),
                            FiniteSpace(ax(:Habitat, HQ)),
                            two_parent([0.75 0.25; 0.25 0.875])))
utilities = Dict{Symbol,AbstractUtility}(
    :Cost1 ⇒ TabularUtility([ax(:Burn1, YN)], [0.0, -1.0]),
    :Cost2 ⇒ TabularUtility([ax(:Burn2, YN)], [0.0, -1.0]),
    :Wildfire ⇒ TabularUtility([ax(:Fuel2, LH)], [0.0, -8.0]),
    :HabitatValue ⇒ TabularUtility([ax(:Habitat, HQ)], [0.0, 10.0]))
nothing # hide
```

3 With a rule each year

If each season had its own rule, deterministic rules would suffice (Kuhn's theorem, proved in the Lean module `Kuhn`), and the closed composite can be optimised directly. The diagram forgets, since `Burn2` does not know `Burn1`, so the exhaustive search solves it.

```
whole = compose(chain...; no_forgetting = false)
separate = optimize(whole, kernels, utilities; backend = ExhaustivePolicySearch())
(value = separate.expected_utility,
 burn1 = policy_table(separate.strategy[:Burn1])[],
 burn2 = policy_table(separate.strategy[:Burn2])[])
```

```
(value = 4.75, burn1 = :no, burn2 = :yes)
```

Leave the first season and burn in the second: worth 4.75.

4 One rule for both seasons

A shared rule must give both seasons the same probability `p` of burning. `shared` ties `Burn2` to the policy of `Burn1`; the strategy gives a policy for `Burn1` only. A deterministic shared rule either never burns or always burns.

```
tie = [[:Burn1, :Burn2]]
burn_with(p) = Strategy(:Burn1 ⇒ StochasticPolicy(:Burn1,
                                                  FiniteKernel(FiniteSpace(),
                                                               FiniteSpace(ax(:
                                                               Burn1, YN)),
                                                               [1 - p, p])))
fixed = shared_policy_search(chain, kernels, utilities; shared = tie, grid = 1,
                             no_forgetting = false)
fixed.value
```

```
-2.0
```

The better fixed rule, burning every year, is worth -2; never burning is worth -3.75. A randomised rule does much better. On a grid of sevenths the search finds it:

```
best = shared_policy_search(chain, kernels, utilities; shared = tie, grid = 7,
                            no_forgetting = false)
(value = round(best.value; digits = 12),
 burn_probability = policy_kernel(best.strategy[:Burn1]).table[2])
```

```
(value = 0.25, burn_probability = 0.5714285714285714)
```

Burning with probability 4/7 each season is worth 0.25 (exactly, up to the rounding of sevenths). This is the absent-minded driver of Piccione and Rubinstein in an ecological setting: when a rule cannot depend on which season it is in, randomising is the only way to burn in about one season out of two. The Lean module `Behaviour` proves the driver's version (`randomising_helps`).

5 Testing a shared rule

The equilibria of the fourth vignette ask whether any one component could gain by changing its own part of the strategy alone. At the shared optimum, each season would: treated as separate decisions, with the same probabilities, each season gains by changing its own action.

```
untied = Strategy(:Burn1 ⇒ best.strategy[:Burn1],
                  :Burn2 ⇒ StochasticPolicy(:Burn2,
                                            FiniteKernel(FiniteSpace(),
                                                         FiniteSpace(ax(:Burn2,
                                                         YN)),
                                                         [3 / 7, 4 / 7])))
policy_regrets(chain, kernels, utilities, untied; no_forgetting = false)
```

```
Dict{Symbol, Float64} with 2 entries:
  :Burn1 => 0.857143
  :Burn2 => 0.642857
```

Both untied regrets are positive. The right test for a shared rule changes the rule towards another one and sums the gains from changing each use alone, each valued in its own season's context. At an optimum that sum is at most zero: the first-order condition of Piccione and Rubinstein's multi-self consistency, proved in the Lean module `SharedPolicy` (`shared_optimum_first_order`). `policy_regrets` with `shared` computes it:

```
[(p = p, regret = policy_regrets(chain, kernels, utilities, burn_with(p); shared =
tie,
                                 no_forgetting = false)[:Burn1])
 for p in (0.0, 2 / 7, 4 / 7, 6 / 7, 1.0)]
```

```
5-element Vector{@NamedTuple{p::Float64, regret::Float64}}:
 (p = 0.0, regret = 14.0)
 (p = 0.2857142857142857, regret = 5.0)
 (p = 0.5714285714285714, regret = 5.551115123125783e-16)
 (p = 0.8571428571428571, regret = 5.999999999999999)
 (p = 1.0, regret = 10.5)
```

The regret vanishes at 4/7, up to rounding, and nowhere else on the grid, and `is_policy_equilibrium` agrees:

```
is_policy_equilibrium(chain, kernels, utilities, best.strategy; shared = tie,
                      no_forgetting = false)
```

```
true
```

The condition is necessary, not sufficient: it is satisfied at every optimum, and a strategy that satisfies it must still be compared with the others, as `shared_policy_search` does. Given evidence, the same functions use the first-order change of the conditional value.

6 A colimit certificate for the composite

`compose` built the two-season diagram as a Catlab pushout of the two seasons along their shared boundary. `export_colimit_certificate` records the reward certificates of the parts and of the composite, under one deterministic strategy, together with Catlab's cocone maps: where each variable, generator and utility of each season went in the composite.

```
σ = separate.strategy
cert = export_colimit_certificate(:compose, season1, season2, kernels, utilities, σ;
                                  no_forgetting = false,
                                  description = "two seasons of prescribed burning")
(operation = cert["operation"], parts = length(cert["parts"]),
 legs = [(variables = leg.variables, generators = leg.generators,
          utilities = leg.utilities) for leg in cert["legs"]])
```

```
(operation = "compose", parts = 2, legs = [(variables = [0, 1], generators = [0, 3],
utilities = [0]), (variables = [0, 1, 2, 3, 4], generators = [1, 2, 4], utilities =
[1, 2, 3])])
```

The first season's two variables land on the composite's first two, and the second season's input variables are glued to them; every generator and utility lands once. The Lean checker `lake exe check_composition` (in the package's `proofs/` directory) accepts the certificate when every reward certificate checks and these maps are a colimit witness: bijective on generators and utilities, preserving labels, tables, parents and scopes, and joining only boundary variables. It then proves, exactly, that the composite's reward kernel is the composite of the seasons' reward kernels. When the checker has been built, this runs it:

```
const JSON3 = CategoricalInfluenceDiagrams.InfluenceDiagrams.JSON3
checker = joinpath(pkgdir(CategoricalInfluenceDiagrams), "proofs", ".lake", "build",
"bin",
                   "check_composition")
if isfile(checker)
    path = joinpath(mktempdir(), "burning.json")
    write(path, JSON3.write(cert))
    print(read(`$checker $path`, String))
else
    println("Build the checker with `lake build check_composition` in proofs/ to run
    it.")
end
```

```
OpenInfluence.ColimitCertificate/v1 (compose): tolerance 1.00000e-12
  part 0: input rows 0, output rows 2, accepted
  part 1: input rows 2, output rows 0, accepted
  composite: input rows 0, output rows 0, accepted
  legs a colimit witness: true
accepted
```

The same certificate can be written for `otimes`, `glue` and `oapply`; for `oapply` the diagram is an undirected wiring diagram whose junctions are the shared variables:

```
uwd = @relation () begin
    first_season(burn1, fuel1)
    second_season(burn1, fuel1)
end
wired = export_colimit_certificate(uwd, chain, kernels, utilities, σ; no_forgetting
= false)
wired["operation"], length(wired["legs"])
```

```
("oapply", 2)
```

7 Summary

- A rule applied in several places is declared with `shared`; with one, the optimum can be randomised (`shared_policy_search`), and here a rule that burns with probability 4/7 beats both fixed rules by more than 2.
- At a shared optimum, each use alone may still want to change; the test is the first-order condition (`policy_regrets`, `is_policy_equilibrium`), proved in Lean.
- A colimit certificate records how a composite was built, and the Lean checker turns that record into an exact statement about the composite's semantics.